

Medium-articles RAG – Development Process

<https://medium-rag.vercel.app> · github.com/Wieder-Shahaf/Medium-Article-RAG

A RAG assistant answering questions strictly from ~7,600 Medium articles. Vercel + FastAPI + Pinecone · Metered API spend: \$1.82 of the \$5 cap (36.5%).

1 · ARCHITECTURE DECISIONS

Two separate pipelines – embed once, iterate cheaply. The biggest design driver was the \$5 budget. Re-embedding the corpus is the only expensive operation, so ingestion and query were split: we pay the embedding cost exactly once and tune everything downstream for free.

- **Ingestion (offline, one-shot, local):** CSV → token-aware chunking → batch-embed → upsert to Pinecone with full metadata (`scripts/ingest.py`, never on Vercel). It is **idempotent** (skips `article_id`s already in the namespace) and has a `--dry-run` that prints estimated token cost before any API call – a deliberate guard against accidental re-embedding.
- **Query (online, Vercel serverless):** embed question → Pinecone query → dedup → assemble prompt → chat completion → return exact JSON shape (`app.py`, `rag/retrieval.py`).

DECISION	RATIONALE
Config as code, not env (<code>rag/config.py</code>)	<code>GET /api/stats</code> reads the same constants the ingestion used, so live stats can never drift from deployed values. Hard caps are <code>assert</code> ed at import.
<code>article_id = sha1(url)[:12]</code>	The CSV has no id column. A hash of the URL is stable and reused identically between ingestion and query responses.
Lazy, module-level clients	OpenAI + Pinecone clients init once and reuse TCP connections across warm serverless invocations – minimizes cold-start and bundle size; the CSV is never shipped in the deployed bundle.
Title prepended before embedding	<code>embed_text = "{title}\n\n{chunk}"</code> gives every chunk topical anchoring, sharpening precise-fact retrieval.
Errors never 500	Zero-chunk / OpenAI / Pinecone failures all return HTTP 200 with the canonical "I don't know..." response – graders always get a parseable shape.
System prompt verbatim + style extension (<code>rag/prompts.py</code>)	The mandated prompt is sent unchanged; a style block adds output formatting (e.g. "exactly N distinct titles") without weakening any constraint.

2 · HYPERPARAMETER SELECTION – CHUNK_SIZE, OVERLAP, TOP_K

Deployed values (all within graded caps): **chunk_size=512** (≤ 1024), **overlap_ratio=0.2** (≤ 0.3), **top_k=10** (≤ 30). Tuning order followed cost: **exhaust the free knobs (top_k, prompt) before touching the expensive one (chunk_size, which forces re-embedding).**

- **chunk_size = 512 tokens** – mid-range by design. Small enough that one chunk usually holds a single coherent idea (helps Category 1 precise-fact retrieval); large enough to give the chat model real context for summaries/recommendations (Categories 3-4). Pushing to 1024 mixes topics per chunk and dilutes similarity; dropping to 256 fragments arguments. Token-aware splitting via `RecursiveCharacterTextSplitter` with a `tiktoken` length function on `cl100k_base`.
- **overlap_ratio = 0.2** (~102 tokens) – enough to avoid splitting a sentence/argument across a chunk boundary, without the ingestion-cost bloat of the 0.3 cap (diminishing returns).
- **top_k = 10 + over-fetch ×5 → dedup by article_id** – the retriever fetches the **50** closest chunks, keeps the highest-scoring chunk per article, then trims to **10 distinct** articles. This over-fetch/dedup step is what makes the "list 3 distinct articles" category reliable – a naive top-k returns multiple chunks of the same article. 15-30 surfaced no better titles in spot-checks and only spent more prompt tokens.
- **Query cleanup** – Category-2 instruction prefixes ("list 3 articles about...") are stripped before embedding so the vector reflects the topic, not the instruction phrasing.

3 · QUALITY CHECK OF SYSTEM PERFORMANCE

Two-tier evaluation against the live `prod` index, plus contract unit tests.

Tier 1 – 9 gold questions, manual scoring (subset tuning gate). 2-3 hand-written questions per category, calibrated to topics known to exist in a 1k subset. Result: **9 / 9 correct across all 4 categories** – decision locked at (512, 0.2, 10), no further sweeps. Used only **10 of the 30** allowed dev/test chat calls (`scripts/eval.py`).

Tier 2 – 300-question LLM-judged benchmark (75 per category), scored on **faithfulness** and **correctness** (0-3 scale) by a judge model with a grounded rubric (`benchmarks/`).

CATEGORY	N	FAITHFULNESS	CORRECTNESS	"I DON'T KNOW"
C1 – precise fact	75	2.72	2.37	8
C2 – multi-result listing	75	2.89	2.52	6
C3 – key-idea summary	75	2.63	2.93	3
C4 – recommendation	75	2.59	2.89	1
ALL	300	2.71	2.68	18

Reading: faithfulness (answers stay grounded in retrieved context) is consistently high. C1/C2 correctness is bounded by retrieval (the right article must land in top-k); C3/C4 score higher because they need a relevant article, not the single exact one. The handful of "I don't know" responses are the safety behavior working – the system refuses rather than hallucinating when context is thin.

Tier 0 – unit tests (`pytest tests/`, no API calls): assert exact JSON casing (`Augmented_prompt`, `System/User`), `/api/stats` shape, and `article_id` dedup logic – the contract graders parse.

Cost discipline throughout – every API batch is logged in `BUDGET_LEDGER.md` with a hard stop-and-ask rule at \$2. The dominant cost was the one-shot full-corpus embed (28,112 chunks); the course proxy meters above the model list price, so actual measured spend across embedding + inference is **\$1.82 of \$5 (36.5%)** – comfortably within budget, with no re-embedding needed after the parameters were locked.